Data Manipulation & Preprocessing

Advanced Machine Learning

Sam Ogden

Learning objectives

- Move between NumPy arrays and PyTorch tensors without friction
- Predict tensor shapes through indexing, reshaping, and combining
- Turn a pandas DataFrame into a clean, model-ready tensor
- Spot preprocessing steps that leak information across splits

From raw data to a model

- Every model starts the same way: **raw data → tensor → model**
- Most of the bugs you'll hit early on are shape bugs, not math bugs
- Today: get fluent moving between numpy, pandas, and torch tensors

NumPy → PyTorch

`numpy.ndarray` vs `torch.Tensor`

What's the same

- Both are n-dimensional arrays of numbers
- Same mental model: shape, dtype, indexing, broadcasting
- Mostly the same method names too

What's different

- torch defaults to `float32`, numpy to `float64`
- torch tensors can live on a GPU; numpy arrays can't
- torch can track gradients if you ask it to (`requires_grad`) — more on this in few days

Creating one looks nearly identical

NumPy

```
1 import numpy as np
2
3 a = np.arange(12)
4 print(a)
```

```
[ 0  1  2  3  4  5  6  7  8  9 10 11]
```

PyTorch

```
1 import torch
2
3 t = torch.arange(12)
4 print(t)
```

```
tensor([ 0,  1,  2,  3,  4,  5,  6,  7,  8,  9, 10,
11])
```

Indexing works the same way

NumPy

```
1 print(a[2:5])
```

```
[2 3 4]
```

PyTorch

```
1 print(t[2:5])
```

```
tensor([2, 3, 4])
```

Reshaping works the same way

NumPy

```
print(a.reshape(3, 4))
```

```
[[ 0  1  2  3]
 [ 4  5  6  7]
 [ 8  9 10 11]]
```

PyTorch

```
print(t.reshape(3, 4))
```

```
tensor([[ 0,  1,  2,  3],
        [ 4,  5,  6,  7],
        [ 8,  9, 10, 11]])
```

Same values, different default dtype

NumPy

```
1 print(np.ones(3).dtype)
```

```
float64
```

PyTorch

```
1 print(torch.ones(3).dtype)
```

```
torch.float32
```

- numpy defaults to `float64` — general scientific computing, precision first
- torch defaults to `float32` — half the memory, and GPUs are much faster at it

`np.arange(12)` and `torch.arange(12)` don't give you the same dtype by default. Be explicit when it matters:

```
1 t32 = torch.arange(12, dtype=torch.float32)
2 print(t32.dtype)
```

```
torch.float32
```

The explicit connection between the two

NumPy → PyTorch

```
shared = torch.from_numpy(a)
shared[0] = 99
print(a[0])
```

```
99
```

PyTorch → NumPy

```
back = t.numpy()
back[0] = -1
print(t[0])
```

```
tensor(-1)
```

Both directions share memory — no copy happens. Change one, change the other. Great for speed, easy to get burned by if you’re not expecting it.

Foreshadowing: `requires_grad`

```
1 w = torch.tensor([1.0, 2.0, 3.0], requires_grad=True)
2 print(w)
```

```
tensor([1., 2., 3.], requires_grad=True)
```

- `requires_grad=True` tells torch to track every operation done on this tensor
- That tracking is what makes automatic differentiation possible
- This is the thing numpy just doesn't do — we'll build directly on it when we get to training loops

Check your understanding

```
x = torch.arange(6, dtype=torch.float64)
```

Predict:

- `x.dtype`
- `x.numpy().dtype`
- What happens to `x` if you modify `x.numpy()` in place?
- Why did we write `dtype=torch.float64` here instead of just letting it default? When might you actually want `float64`?

Answers:

- `x.dtype` → `torch.float64`
- `x.numpy().dtype` → `float64` — numpy mirrors torch's dtype here
- Modifying `x.numpy()` in place also modifies `x` — they share memory
- `float64` earns its cost when precision loss would actually break something — accumulating many small numbers, ill-conditioned linear algebra, scientific/financial computation. Rare in DL training itself; more common in preprocessing or evaluation code

Shape manipulation

The shape is the *contract*

- Shape encodes what the data *means*: which axis is batch, which is channel, which is a feature
- Most early bugs in this class are shape bugs, not math bugs
- Before you run a line, predict its output shape — that habit is worth more than memorizing the API

Reshape: reinterpreting the same data

```python
x = torch.arange(1, 10)
print(x)
print(x.shape)

square = x.reshape(3, 3)
print(square)
print(square.shape)
```

```
tensor([1, 2, 3, 4, 5, 6, 7, 8, 9])
torch.Size([9])
tensor([[1, 2, 3],
        [4, 5, 6],
        [7, 8, 9]])
torch.Size([3, 3])
```

- Same 9 numbers, **same underlying memory**, it's just a different shape way of thinking about the data.
  - Think a 2D array
- Reshape doesn't move or copy data; it changes how the data is *read*

The same idea, at model scale

```
x = torch.arange(64 * 1 * 28 * 28).reshape(64, 1, 28, 28)
print(x.shape)

flat = x.reshape(x.shape[0], -1)
print(flat.shape)
```

```
torch.Size([64, 1, 28, 28])
torch.Size([64, 784])
```

- A batch of 64 grayscale 28×28 images, flattened to feed an neural network
- `-1` means “figure this dimension out for me”
  - useful when you don’t want to hardcode it

`cat` joins along an existing axis, `stack` makes a new one

`torch.cat`

```
 1  a1 = torch.zeros(2, 3)
 2  a2 = torch.ones(2, 3)
 3  print(torch.cat([a1, a2], dim=0).shape)
```

```
torch.Size([4, 3])
```

`torch.stack`

```
 1  print(torch.stack([a1, a2], dim=0).shape)
```

```
torch.Size([2, 2, 3])
```

- `cat` on two `(2, 3)` tensors along `dim=0` → `(4, 3)` — no new axis, just longer
- `stack` on the same tensors → `(2, 2, 3)` — a brand new axis is created

`squeeze` / `unsqueeze`: adding or removing size-1 axes

```
example = torch.arange(5)
print(example.shape)

batched = example.unsqueeze(0)
print(batched.shape)
```

```
torch.Size([5])
torch.Size([1, 5])
```

- A model is built to expect a **batch** of examples, shape `(batch, features)` — even when you only have one example to give it
- `unsqueeze(0)` makes a single example *look like* a batch of one, so it fits the model's expected shape
- `squeeze(0)` undoes it — strip the batch axis back off once you're done

Broadcasting: how PyTorch lines up mismatched shapes

Line the two **shapes** up on their **right** edge. Walk right to left, axis by axis. At each axis, you need either the same number, or one of them to be `1` (which gets stretched to match).

```
shape(batch):  (4, 3)
shape(bias) :     (3,)
              ------
shape(result): (4, 3)
```

- These rows are *shapes*, not the actual numbers in the tensors
- Rightmost axis: `3` vs `3` → match
- Next axis: `4` vs *nothing* → treat the missing axis as `1`, stretch it to `4`

Same rule, in code

```
batch = torch.ones(4, 3)
bias = torch.tensor([1.0, 2.0, 3.0])
print((batch + bias).shape)
```

```
torch.Size([4, 3])
```

- `bias` (shape `(3,)`) gets added to every one of the 4 rows in `batch`
- No loop, no manual copying — broadcasting does the repeating for you

When broadcasting can’t save you

```
shape(batch):  (4, 3)
shape(wrong):      (2,)
               ------
shape(result):     ✗
```

- Again, these rows are *shapes* — `batch` and `wrong` are still just tensors of numbers
- Rightmost axis: `3` vs `2` → neither matches, and neither is `1` → broadcasting gives up here

```
1 batch = torch.ones(4, 3)
2 wrong = torch.tensor([1.0, 2.0])
3 batch + wrong
```

```
---------------------------------------------------------------------------
RuntimeError                              Traceback (most recent call last)
Cell In[19], line 3
      1 batch = torch.ones(4, 3)
      2 wrong = torch.tensor([1.0, 2.0])
----> 3 batch + wrong

RuntimeError: The size of tensor a (3) must match the size of tensor b (2) at non-singleton dimension 1
```

Predict before you run

```
x = torch.arange(24).reshape(4, 6)
y = x[:, :3]
z = torch.stack([y, y], dim=0)
w = z.reshape(z.shape[0], -1)
```

Predict the shape of `x`, `y`, `z`, and `w` — in that order — before running anything.

```
1 x = torch.arange(24).reshape(4, 6)
2 y = x[:, :3]
3 z = torch.stack([y, y], dim=0)
4 w = z.reshape(z.shape[0], -1)
5 print(x.shape, y.shape, z.shape, w.shape)
```

```
torch.Size([4, 6]) torch.Size([4, 3]) torch.Size([2, 4, 3]) torch.Size([2, 12])
```

Pandas refresher

You’ve already used this — fast recap

- A DataFrame is a table: rows are examples, columns are features
- Today’s version skips the syntax tour and goes straight to what you’ll actually reach for

From disk to a DataFrame

```
1 import pandas as pd
2
3 df = pd.DataFrame({
4     "age": [22.0, 35.0, None, 58.0],
5     "fare": [7.25, 71.83, 8.05, 26.00],
6     "sex": ["male", "female", "female", "male"],
7     "survived": [0, 1, 1, 0],
8 })
9 df
```

	age	fare	sex	survived
0	22.0	7.25	male	0
1	35.0	71.83	female	1
2	NaN	8.05	female	1
3	58.0	26.00	male	0

What you check first

Did it parse right?

```
df.dtypes
```

```
age          float64
fare         float64
sex              str
survived       int64
dtype: object
```

What’s missing?

```
df.isna().sum()
```

```
age         1
fare        0
sex         0
survived    0
dtype: int64
```

`describe()`: a fast sanity check on the numbers

```
1  df.describe()
```

	age	fare	survived
count	3.000000	4.000000	4.00000
mean	38.333333	28.282500	0.50000
std	18.230012	30.294745	0.57735
min	22.000000	7.250000	0.00000
25%	28.500000	7.850000	0.00000
50%	35.000000	17.025000	0.50000
75%	46.500000	37.457500	1.00000
max	58.000000	71.830000	1.00000

- Skim for the obviously wrong: negative ages, a suspicious max, a count that doesn’t match the row total

Selecting rows and columns

```
1 df["age"]
```

```
0    22.0
1    35.0
2     NaN
3    58.0
Name: age, dtype: float64
```

```
1 df.loc[df["sex"] == "female"]
```

	age	fare	sex	survived
1	35.0	71.83	female	1
2	NaN	8.05	female	1

Worth knowing: `groupby`

```
1 df.groupby("sex")["survived"].mean()
```

```
sex
female    1.0
male      0.0
Name: survived, dtype: float64
```

- A quick sanity check before you build features: does this column even look related to what you're predicting?
- We won't lean on this heavily this semester — just worth knowing it's there

Creating Tensors from DataFrames

Can we just do `torch.tensor(df.values)`?

`df` still has a string column (`sex`) and a missing value (`age`) in it. Let's try converting it straight to a tensor anyway:

```
1 print(df.values.dtype)
```

```
object
```

```
1 torch.tensor(df.values)
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[29], line 1
----> 1 torch.tensor(df.values)

TypeError: can't convert np.ndarray of type numpy.object_. The only supported types are: float64, float32, flo
```

Mixed strings and numbers force numpy into `dtype=object` — an array of arbitrary Python objects, not numbers. Torch has no idea how to turn that into a tensor. You have to clean the DataFrame *before* converting.

Step 1: handle missing values

```
1 df["age"] = df["age"].fillna(df["age"].median())
2 df
```

	age	fare	sex	survived
0	22.0	7.25	male	0
1	35.0	71.83	female	1
2	35.0	8.05	female	1
3	58.0	26.00	male	0

- Decide this deliberately — fill with median? drop the row? A tensor has no concept of "missing," so this decision has to happen in pandas, before conversion

Step 2: turn categories into numbers

```
df["sex_male"] = (df["sex"] == "male").astype(int)
df[["age", "fare", "sex_male", "survived"]]
```

	age	fare	sex_male	survived
0	22.0	7.25	1	0
1	35.0	71.83	0	1
2	35.0	8.05	0	1
3	58.0	26.00	1	0

- A tensor is just numbers — no strings allowed
- Here, a simple 0/1 map is enough; with more than two categories you'd reach for `pd.get_dummies`

Step 3: assemble the tensors

```
 1 features = df[["age", "fare", "sex_male"]].values
 2 labels = df["survived"].values
 3
 4 X = torch.tensor(features, dtype=torch.float32)
 5 y = torch.tensor(labels, dtype=torch.float32)
 6
 7 print(X.shape, y.shape)
 8 print(X)
```

```
torch.Size([4, 3]) torch.Size([4])
tensor([[22.0000,  7.2500,  1.0000],
        [35.0000, 71.8300,  0.0000],
        [35.0000,  8.0500,  0.0000],
        [58.0000, 26.0000,  1.0000]])
```

- `X` is `(examples, features)`, `y` is `(examples,)` — this is the shape every model in this class will expect

The catch: fit preprocessing on train only

```
raw = pd.DataFrame({
    "age": [22.0, 35.0, None, 58.0],
    "fare": [7.25, 71.83, 8.05, 26.00],
})

train, test = raw.iloc[:3], raw.iloc[3:]

age_median = train["age"].median()      # fit on train only
train = train.fillna({"age": age_median})
test = test.fillna({"age": age_median})  # reuse the train statistic
```

Never call `.fillna(raw["age"].median())` on the whole dataset before splitting

- that lets test-set information leak into training.

Any statistic you fit (mean, median, category list, min/max for scaling) gets calculated on train, then reused as-is on validation and test.

Takeaways

- Tensors and numpy arrays are basically the same idea — shape, dtype, indexing, and broadcasting all carry over
- Track shapes deliberately: reshape doesn't move data, `cat`/`stack`/`squeeze`/`unsqueeze` compose predictably, and broadcasting has exactly one rule
- Getting from a DataFrame to a tensor means handling missing values and categoricals explicitly, then fitting any preprocessing statistic on train only

[Backup] Autograd teaser

`requires_grad` in action

```
w = torch.tensor([2.0], requires_grad=True)
loss = (w - 5) ** 2
loss.backward()
print(w.grad)
```

```
tensor([-6.])
```

- torch tracked every operation from `w` to `loss`
- `.backward()` computes `d(loss)/d(w)` automatically — no calculus done by hand
- That gradient is exactly what an optimizer uses to update `w`. This is the entire mechanism training runs on — we'll spend real time here soon